Perfect. Here's a fully annotated Lucy folder & logic flow, showing how each submodule interacts with aura_core, agents, and the dashboard. I'll keep it fully clear and separated for your project blueprint.

Lucy Folder Structure

```
lucy/
├── __init__.py
├── planner.py                # Core reasoning & action planning
├── interface_manager.py      # Routes queries, commands, and agent feedback
├── eagle_eyes.py             # Real-time monitoring of agent execution
├── reward_memory.py          # Stores historical states, actions, and rewards
├── system_guardian.py        # Ensures actions stay within governance constraints
├── task_executor.py          # Executes planned subtasks safely
├── agent_adapters/           # Wraps external agents for Lucy orchestration
│   ├── energy_agent.py
│   ├── policy_agent.py
│   └── ai_agent.py
├── voice_interface.py        # Handles speech output & alerts
├── visual_observer.py        # Monitors visual simulation outputs
└── data_logger.py            # Writes all Lucy interactions & decisions to audit logs
```

Lucy Logic Flow

```
flowchart TD
    %% Lucy Core
    P[Planner] -->|Requests simulation state| AC[aura_core]
    AC -->|Returns KTE, CEI, TT1, Coherence| P
    P -->|Generates action proposals| TE[Task Executor]

    %% Interface
    IM[Interface Manager] -->|Routes human/user queries| P
    IM -->|Routes agent feedback| P
    P -->|Sends instructions to agents| A[Agents]

    %% Monitoring
    EE[Eagle Eyes] -->|Observes agent steps| A
    EE -->|Reports confidence / corrections| P
```

```
%% Memory / Learning
RM[Reward Memory] -->|Stores states & actions| AC
AC -->|Historical states for reasoning| RM

%% Outputs
VI[Voice Interface] -->|Explains changes / alerts| Dashboard
VO[Visual Observer] -->|Visualizes coherence, KTE, Monte Carlo| Dashboard

%% Safety / Governance
SG[System Guardian] -->|Intercepts unsafe actions| TE
SG -->|Consults FORB-Registry| AC
```

How Calculations Are Deliberately Controlled

1. All equations live in aura_core. Lucy never calculates KTE, CEI, TT1, or Coherence herself.

2. Agents only propose parameters (T_i, A_i, P_i). Lucy collects them, validates via System Guardian, then sends to aura_core.

3. No agent or Lucy module can bypass governance — Eagle Eyes and System Guardian act as real-time “safety nets.”

4. Reward Memory + Planner can simulate potential outcomes for proposed actions before submitting them to aura_core.

5. Dashboard / Voice / Visual only visualize results; they cannot alter the simulation.

Integration of Lucy into Planetary Blueprint

Input Layer: Lucy receives agent proposals and human queries.

Processing Layer: Planner + Interface Manager + Eagle Eyes → simulate safe decisions.

Execution Layer: Task Executor → sends only governance-approved proposals to aura_core.

Output Layer: Reward Memory + Voice/Visual interfaces → explain outcomes and provide real-time alerts.

This setup ensures cohesion, safety, and complete auditability, while allowing real-time multi-agent orchestration without ever crossing forbidden intervention boundaries.

If you want, I can next draw the full planetary project blueprint folder structure, combining aura_core, Lucy, agents, dashboard, sandbox, and risk management, fully annotated and ready for implementation.

Do you want me to do that next?